Infant Cry State Recognition

Project plan and architecture: what will be built, how it will be evaluated, and in what order

Status: approved 28 Oct 2026, with notebook 07 moved to Google Colab (section 7.1). Only notebook 01 is built

Date: 28 Oct 2026

Repo: techatanu/Infant-State-Recognition-System @ 9b5bbae

> **How to read this.** Sections 1–3 fix the goal and the ground rules. Sections 4–6 describe the architecture and the evaluation protocol, which is the part that decides whether any reported number can be trusted. Section 7 goes notebook by notebook. Section 8 is the build order (one commit per chunk). Section 10 records the decisions.

1. Goal and scope

Task. Given a ~7 second recording of a baby crying, predict which of 8 states the parent reported: *belly_pain, burping, cold_hot, discomfort, hungry, lonely, scared, tired*. These are needs or states rather than emotions in the psychological sense, and they were entered by parents in a crowd-sourced app, so the labels themselves are noisy.

Approach. First classical machine learning on hand-engineered audio features, then deep learning on spectrograms, both evaluated with *exactly the same* data, splits and metrics so the comparison is fair.

Success means

- A model that beats the "always *hungry*" baseline on **balanced accuracy** and **macro-F1**, with uncertainty intervals that do not overlap the baseline.
- A reader can follow every step from raw files to final numbers, and every number in the README traces to a notebook output.
- Limitations stated plainly, including which classes cannot be estimated with this data.

Out of scope (unless you ask)

- Medical or diagnostic claims.
- Cry detection (deciding whether a sound is a cry at all).
- A deployed demo app. It could be added after notebook 08 if the results justify one.
- Collecting new data.

2. What the data really contains (from notebook 01)

Class	Files	Generated copies	Label mismatch	Clean clips	Sources (babies)	Stored format
hungry	382	0	0	**382**	205	100% 8 kHz
tired	28	0	0	**28**	17	86% 8 kHz
discomfort	30	0	9	**21**	14	95% 8 kHz
belly_pain	16	0	0	**16**	7	100% 8 kHz
lonely	11	0	0	**11**	8	100% 44.1 kHz
burping	18	9	0	**9**	8	89% 8 kHz
cold_hot	7	0	0	**7**	3	100% 44.1 kHz
scared	20	15	0	**5**	4	100% 44.1 kHz
total	**512**	**24**	**9**	**479**	**226**	

Four facts drive the whole design:

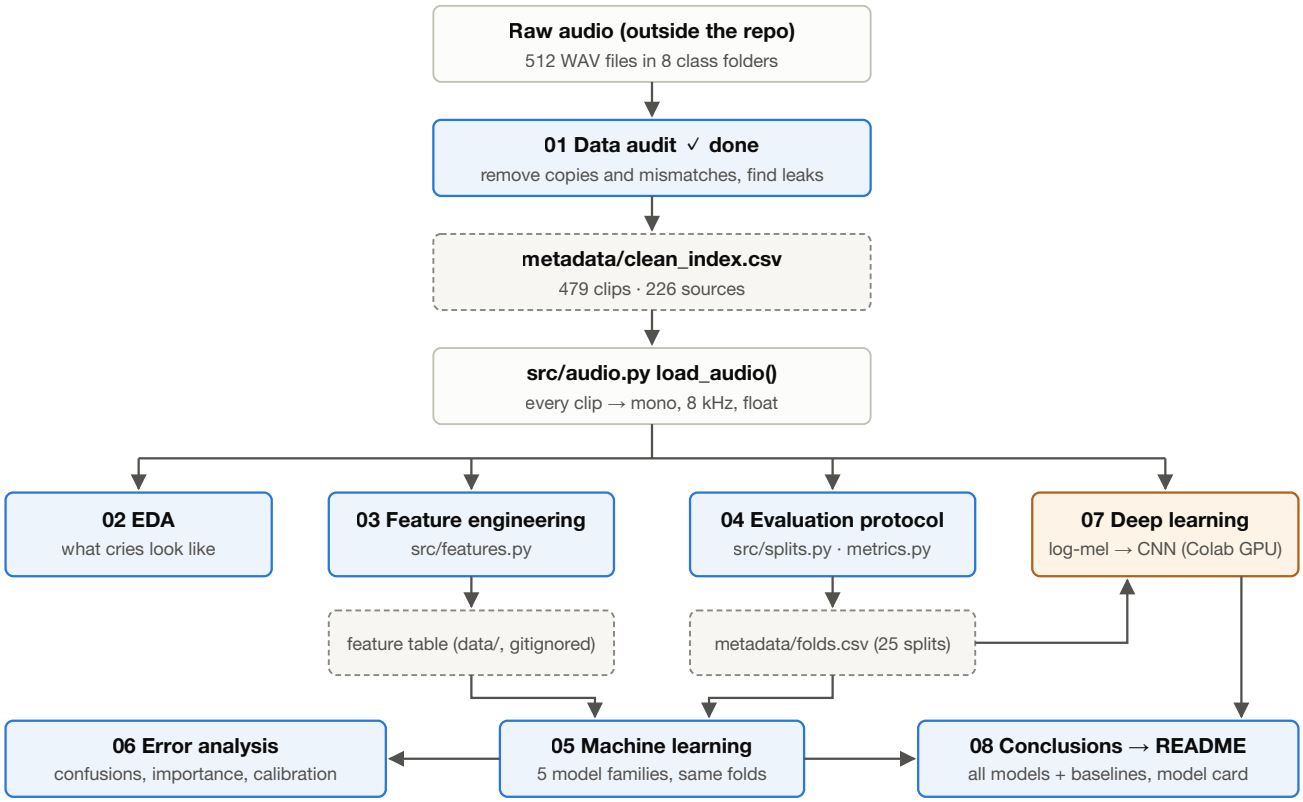
1. **Extreme imbalance.** 80% of clips are *hungry*. Always answering *hungry* already scores accuracy 0.797 (balanced accuracy 0.125, macro-F1 0.111). Accuracy therefore cannot be the headline metric.
2. **Tiny rare classes.** *scared* (4 babies), *cold_hot* (3) and *burping* (8) are too small for reliable per-class scores. Every per-class number will carry its baby count.
3. **Same baby, many clips.** 116 of 226 sources contribute several clips. Splitting clips at random would let the model recognise a baby rather than a cry type, so **every split is grouped by source**.
4. **File format follows the labels.** 44.1 kHz stereo files are really phone-quality audio (identical channels, nothing above ~4.5 kHz), but the format still lines up with classes. All audio is converted to **8 kHz mono**, and notebook 03 tests whether any format signature survives.

3. Ground rules

Rule	What it means in practice
No number without a run	Every metric in a notebook or README is printed by code that ran on the real data. Anything that could not be run is marked TODO, never estimated.
One clean dataset	All modelling reads `metadata/clean_index.csv` (479 clips). Generated copies and mismatched files never come back.
Grouped by baby, always	No source appears on both sides of any split: outer folds, inner tuning folds, early-stopping splits.
One audio format	All audio enters the pipeline as 8 kHz mono through a single function (`src/audio.py`).
No peeking	Every choice (hyperparameters, decision thresholds, epochs, feature selection) is made on training folds only, never on the fold being scored.
Same protocol for ML and DL	Identical saved folds (`metadata/folds.csv`), metrics and reporting code for every model.
Report everything tried	The model menu is declared before running, and every model's score is shown, not only the best one.
Reusable code is tested	Functions used by more than one notebook live in `src/` with pytest tests on synthetic signals.

4. Architecture

4.1 Data flow



Blue: notebooks I build and run here. Orange: notebook you run on a Google Colab GPU (section 7.1). Dashed: files produced and read by notebooks. Each model notebook writes its scores to `results/`, which 06 and 08 read.

4.2 Repository layout (target)

```
Infant-State-Recognition-System/
├── README.md                  project summary; every number links to a notebook
├── requirements.txt           notebooks 01–06, 08 (numpy, scipy, pandas, scikit-learn, matplotlib)
├── requirements-dl.txt        notebook 07 extras (Colab already ships torch)
├── .env.example               BABYCRY_DATA_DIR=/path/to/dataset   (.env itself is gitignored)
├── metadata/
│   ├── clean_index.csv        479 clips used for modelling            (from 01)
│   └── folds.csv              clip → repeat, fold for all 25 splits    (from 04)
├── notebooks/
│   ├── 01_data_audit.ipynb    ✓
│   ├── 02_eda.ipynb
│   ├── 03_feature_engineering.ipynb
│   ├── 04_evaluation_protocol.ipynb
│   ├── 05_machine_learning.ipynb
│   ├── 06_error_analysis.ipynb
│   ├── 07_deep_learning.ipynb
│   └── 08_conclusions.ipynb
├── scripts/
│   └── make_colab_bundle.py   zip of the 479 clean clips at 8 kHz mono, for upload to Drive
├── src/                       code shared by more than one notebook
│   ├── audio.py               paths, class list, load_audio() → 8 kHz mono
│   ├── descriptors.py         cry/pause segmentation, YIN pitch, spectral centroid
│   ├── features.py            framing, mel filterbank, MFCC, clip-level feature vector
│   ├── splits.py              repeated StratifiedGroupKFold, inner grouped splits
│   ├── metrics.py             balanced acc, macro-F1, per-class table, cluster bootstrap
│   ├── cnn.py                 log-mel input, CNN model, training loop (notebook 07)
│   └── viz.py                 one plotting style for all figures
├── tests/                     pytest on synthetic audio, no dataset needed
├── results/                   small CSV/JSON score files written by notebooks 05 and 07
└── docs/PROJECT_PLAN.pdf      this document
```

4.3 Shared code and its tests

Module	Responsibility	Tested by (synthetic signals, no dataset)
`audio.py`	Read WAV, average channels, resample to 8 kHz, scale to [-1, 1]	44.1 kHz stereo tone → correct length, mono, same frequency peak; clean index contains no generated or mismatched file
`descriptors.py`	Frame RMS, cry/pause segments, YIN f0, spectral centroid	Known-pitch harmonic tones recovered within 2%; noise mostly unvoiced; tone bursts counted correctly
`features.py`	Mel filterbank, MFCC + deltas, spectral, pitch, rhythm features → one vector per clip	Fixed length, no NaN on edge cases (near-silent, very short); a pure tone peaks in the right mel band
`splits.py`	Build and load the 25 outer splits and inner splits	No source on both sides; every clip tested once per repeat; reproducible from seed
`metrics.py`	All metrics, per-class table with baby counts, cluster bootstrap CIs, paired comparison	Hand-computed examples; macro-F1 always averages all 8 classes
`cnn.py`	Spectrogram input, model, training with grouped early stopping	Output shape, one training step lowers loss (CPU, tiny synthetic batch; runs here if torch is present, otherwise in Colab)

5. Evaluation protocol

This section decides whether results can be believed, so it is fixed *before* any model is trained.

5.1 Splits

- **Outer loop:** `StratifiedGroupKFold`, 5 folds, grouped by source, repeated with 5 different seeds, which gives **25 train/test splits**. In each repeat every clip is predicted exactly once by a model that never saw its baby. The splits are saved to `metadata/folds.csv` and reused unchanged by every model, ML and DL.
- **Inner loop** (inside each outer training part only): grouped 3-fold CV for hyperparameter tuning, decision-threshold tuning and, for the CNN, early stopping. The outer test fold is never used for any choice.
- **Why no single held-out test set:** a 20% hold-out would contain about one *scared* baby and one *cold_hot* baby, so its rare-class scores would be a coin flip. Repeated grouped CV scores every clip and shows how much results move between splits. The cost is that choosing the best model from the menu uses the same CV, so the menu is kept small and fixed in advance (decision D1 in section 10).

5.2 Metrics

Metric	Role
Balanced accuracy	Primary. The mean recall over the 8 classes, so each class counts equally. Always-*hungry* = 0.125.
Macro-F1 (all 8 labels)	Primary. Averaged over all 8 classes even when a class gets no prediction, so failing classes are never silently dropped.
Per-class precision / recall / F1	Always shown with the number of clips *and* babies. Classes with fewer than 5 babies are flagged "not reliably estimable".
Confusion matrix	Shown as counts and as row-normalised recall.
MCC, accuracy	Secondary. Accuracy is always printed next to the 0.797 baseline.
Log-loss, Brier, top-3 accuracy	Probability quality. These matter because a demo would show the top 3 guesses.

Computation. In each repeat, predictions from the 5 test folds are pooled (one prediction per clip) and the metrics are computed once over all 479 clips. Results are reported as mean ± std across the 5 repeats. **Uncertainty:** 95% intervals from a *cluster bootstrap* that resamples babies, not clips, because clips from one baby are not independent. **Model comparison:** paired cluster bootstrap of the difference in balanced accuracy, using the same resamples for both models.

5.3 Baselines every model must beat

- **Majority:** always *hungry*.
- **Prior-random:** guess classes at random in proportion to their training frequency.
- **Clip-level CV, shown once in 04 only:** the same simple model with ungrouped folds. It shows how much leakage inflates scores and is never used as a result.

6. Handling the imbalance

Three levers, each tested as an explicit option rather than assumed to help:

1. **Class weights in the loss:** none, inverse-frequency ("balanced"), or square-root inverse frequency (milder; the full inverse makes a *scared* clip count about 76 times as much as a *hungry* one: 382 vs 5 clips).
2. **Decision rule:** divide predicted probabilities by the class priors raised to a power τ, with τ tuned on inner-loop predictions. This corrects a model that never predicts rare classes, or one that predicts them too often.

3. **No oversampling of copies.** SMOTE or duplicating rare clips would bring back the "copy of a training clip" problem the audit just removed. Augmentation is used only in DL, on training folds only.

7. Notebooks

01 · Data audit done · commit 9b5bbae

Can the raw files be trusted?

Indexes all 512 files; removes 24 generated copies, 9 label-mismatched files and 1 byte-identical duplicate; documents the format confound and the per-baby structure; computes the majority baseline; writes `clean_index.csv`.

02 · Exploratory data analysis I run

What do the cries look like, and do the classes actually differ?

- One example per class: waveform with detected cry bursts, and spectrogram (0–4 kHz) with the pitch track overlaid.
- How descriptors are computed, explained in plain language: loudness envelope, cry/pause segmentation, YIN pitch.
- Per-class distributions of interpretable descriptors: median pitch, pitch variability (semitones), share of time crying, bursts per second, burst and pause length, spectral centroid. Plotted as one point per baby.
- Statistical tests on per-baby values (to avoid counting one baby many times): Kruskal–Wallis across classes with effect size and Holm correction, and *hungry* vs the rest.
- Average spectrum per class (small multiples against the all-clips average).
- "Every baby has its own voice": intra-class correlation of descriptors within *hungry* babies with several clips. This shows why grouping by baby is necessary.
- **Findings** section written only from the printed outputs.

03 · Feature engineering I run

How do we turn 7 seconds of audio into numbers a model can use?

- Built step by step and shown visually: framing (25 ms / 10 ms, Hann), power spectrum, mel filterbank (0–4 kHz), log-mel, MFCC via DCT, deltas.
- Feature groups: **A** log-mel band statistics · **B** MFCC 1–20 + Δ + ΔΔ (mean, std) · **C** spectral shape (centroid, bandwidth, roll-off, flatness, flux) · **D** pitch and voicing (median, IQR, range, voiced share, frame-to-frame change) · **E** rhythm (cry share, bursts/s, burst and pause durations) · **F** energy dynamics. Absolute loudness is excluded because it depends on the phone.
- Statistics are computed over cry frames only, so pauses don't dilute them.
- **Format-leak test:** can a grouped-CV classifier predict the original file format (8 vs 44.1 kHz) from the features? If yes: band-limit to 150–3800 Hz before feature extraction and/or drop the affected features, then test again.
- Feature understanding (effect sizes, redundancy) is for explanation only. Any feature *selection* happens inside the CV.
- Final feature count is whatever the notebook prints. It is not fixed in advance.

04 · Evaluation protocol and baselines I run

How do we measure fairly, and how noisy is the measurement?

- Builds and saves the 25 grouped splits; shows per-fold class and baby counts (including folds with no *scared* baby).
- Metric definitions with small worked examples; why accuracy misleads here.
- Baselines (majority, prior-random) on the saved folds.
- Leakage demonstration: one simple model scored with clip-level vs grouped folds.
- How much scores vary between repeats: the noise floor any model difference must exceed.

05 · Machine learning I run

Which classical model, with which imbalance handling, works best under the protocol?

Model (fixed menu)	Why it is on the list	Tuned in inner loop
Logistic regression (multinomial, L2)	Simple, well-calibrated reference	C, class weight
k-nearest neighbours	Shows whether similar-sounding cries share labels	k, distance weighting
SVM (RBF kernel)	Strong on small tabular feature sets	C, γ, class weight
Random forest	Non-linear, robust to scaling	depth, min leaf, class weight
Histogram gradient boosting	Usually the strongest tabular model	learning rate, depth, sample weights

- Scaling inside the pipeline only for models that need it (LR, kNN, SVM).
- Each model with and without the tuned prior-correction decision rule.
- Feature-group ablation for the best model: B only → +C → +D → +E/F, showing which information helps.
- Writes `results/ml_scores.csv` and out-of-fold probabilities for 06 and 08.

06 · Error analysis and interpretability I run

Where and why does the best ML model fail?

- Confusion matrix; per-class precision/recall with baby counts; which classes are confused with which.
- Grouped permutation importance by feature group: what the model actually relies on.
- Calibration: reliability diagram; does confidence separate right from wrong (AUROC)?
- Per-baby performance; any effect of recorded age or gender; residual format effect.
- A table of the most confident mistakes (filenames) so you can listen to them.

07 · Deep learning you run on Colab · I write and fix

Does a CNN on spectrograms beat hand-made features under the same protocol?

- Input: 64-band log-mel, 0–4 kHz, 25 ms / 10 ms frames, fixed 7 s (crop or pad).
- Model: small CNN (convolution blocks → global pooling → dropout → linear), sized for 479 clips.
- Training: AdamW; square-root class-weighted loss vs a class-balanced sampler; augmentation on training folds only (time shift, low-level noise, SpecAugment). Pitch shift only as an ablation, because pitch may be one of the cues.
- Early stopping on a grouped validation split carved from the outer *training* part. This fixes the earlier project's optimistic bias of choosing the epoch on the scored fold.
- Same 25 splits, read from `metadata/folds.csv`. If all 25 do not fit in the Colab session budget, run one repeat (5 splits) and say so in the results.
- *Optional 07b:* pretrained audio embeddings (e.g. PANNs, YAMNet or BEATs) with logistic regression on the same splits. Colab has internet access, so the weights can be downloaded there. These models expect 16 or 32 kHz input, so the 8 kHz audio is upsampled; it still contains nothing above 4 kHz, which is stated next to the result.

7.1 Where notebook 07 runs: Google Colab

Researched 28 Oct 2026. Google's official Colab FAQ is the source for limits and behaviour. The sign-up page needs a login, so prices come from secondary sources and must be checked on the sign-up page before paying anything.

Option	What you get	Fit for this project
Colab, free tier	GPU when available (usually a T4, but the FAQ says types "vary over time"). Sessions last at most 12 h, idle runtimes time out, and limits are dynamic and unpublished.	**Chosen.** The input is small (479 clips of 7 s), so a T4 is enough. The workflow below survives disconnects.
Colab Pay As You Go	Compute units bought once, reported as about $9.99 per 100 units, expiring after 90 days. You fall back to free-tier rules when they run out.	Fallback only, if the free tier keeps refusing a GPU. No subscription.
Colab Pro / Pro+	Monthly units, faster GPUs, more memory. Pro+ adds background execution up to 24 h. Reported at about $9.99 and $49.99 per month.	Not needed. A small CNN on 479 clips does not need an A100 or background runs.
Kaggle Notebooks	Free P100 or 2×T4. Third-party sources report about 30 GPU hours a week and 9–12 h sessions (not verified on Kaggle's own docs).	A good free backup with the same notebook. The only change is the data path.

How the Colab run works (all of this is written into notebook 07, so you open it and run all cells):

1. **Data, once:** `scripts/make_colab_bundle.py` runs here and writes a zip of the 479 clean clips, already 8 kHz mono, which you upload to your Google Drive. Audio never goes into git.
2. **Setup cells:** select Runtime → T4 GPU; mount Drive; copy the zip to the VM's local disk and unzip it (reading many small files straight from Drive is slow and can time out, per the FAQ); clone this repo for `src/` and `metadata/`; print the GPU name and the torch version.
3. **Code from the repo:** the notebook imports `src/cnn.py` and `src/metrics.py`, so Colab and this machine use the same splits and metrics. If the repo is private, cloning uses a read-only token stored in Colab's Secrets panel, never in the notebook.
4. **Resumable:** after each split, the out-of-fold predictions are written to a results folder on Drive. On reconnect, finished splits are skipped. That covers the 12 h cap and idle timeouts.

5. **Back to the repo:** you download the executed notebook and `results/cnn_*.csv`, put them in the repo, and I check them and run 06/08 on them here. GPU training is not bit-for-bit reproducible, so seeds are fixed and the spread over repeats is reported.
6. **CPU fallback:** the notebook picks a GPU if there is one and otherwise runs on CPU, so it also works on Kaggle or locally.

Sources: Google Colab FAQ (research.google.com/colaboratory/faq.html) and developers.google.com/colab for limits, Drive and background execution. Prices and Kaggle quotas come from secondary write-ups found by web search on 28 Oct 2026 (aquanode.io, thundercompute.com, spheron.network, aimultiple.com) and are not official.

---

08 · Conclusions I run

What can honestly be claimed?

- One table: every model and baseline, primary metrics with cluster-bootstrap intervals and paired comparisons.
- Per-class results with reliability flags; what is learnable with this data and what is not.
- Limitations (label noise, tiny classes, single dataset, possible recording confounds) and what data would help.
- A short model card; the README is rewritten from this notebook's outputs.

8. Build order (one commit per chunk)

After each chunk: run the tests and execute the notebook end to end in a fresh kernel, stage with `git add -A`, show `git status`, then stop and give you the commit command. The next chunk starts only after you say "done" and I have checked `git log`.

#	Contents	Verified by	Suggested commit message
0	This plan (`docs/PROJECT_PLAN.pdf`), after your approval	your review	Add project plan
1	`src/audio.py`, `descriptors.py`, `viz.py` + tests; `02_eda.ipynb`	pytest + full run	Add audio loading, cry descriptors and EDA notebook
2	`src/features.py` + tests; `03_feature_engineering.ipynb`	pytest + full run	Add feature engineering notebook and feature extraction
3	`src/splits.py`, `metrics.py` + tests; `04_evaluation_protocol.ipynb`; `folds.csv`	pytest + full run	Add grouped evaluation protocol and baselines
4	`05_machine_learning.ipynb`; `results/ml_*`	full run	Add classical ML models under grouped CV
5	`06_error_analysis.ipynb`	full run	Add error analysis of the best ML model
6	`src/cnn.py`, `requirements-dl.txt`, `scripts/make_colab_bundle.py`, `07_deep_learning.ipynb` (Colab-ready, unexecuted)	bundle built here; your Colab run	Add CNN notebook on the shared protocol
6b	07 with your Colab outputs and `results/cnn_*`; fixes from your error messages	your Colab run	Add CNN results
7	`08_conclusions.ipynb`; final README	full run	Add conclusions and final README

9. Risks and how they are handled

Risk	Mitigation
Rare classes (3–8 babies) give unstable scores	Report baby counts and intervals; flag classes under 5 babies; never headline a single rare-class number.
Recording-format signature survives resampling	Explicit leak test in 03; band-limiting and re-test; residual check in 06.
Noisy parent labels cap achievable accuracy	Stated as a limitation; no claims about a "ceiling".
Over-tuning to the CV by trying many things	Fixed model menu, nested tuning, every run reported, paired tests for differences.
Hand-built features contain a bug	Unit tests on signals with known answers; visual checks in notebooks; optional librosa cross-check cell if librosa is installed in your environment.
This machine's sandbox cannot install packages (pip blocked)	Notebooks 01–06 and 08 use only libraries already in your anaconda env. Notebook 07 runs on Colab, which ships PyTorch.
CNN training time is unknown	Measured on the first split and printed; fall back to one repeat if needed, stated in the results.
Colab disconnects, has no free GPU, or changes its limits	Per-split checkpoints on Drive and resume; Pay As You Go or Kaggle as fallback; the notebook also runs on CPU.
Colab and local code drift apart	07 clones the repo and imports `src/`; it prints the commit hash it ran on, and that hash is stored with the results.

10. Decisions (approved 28 Oct 2026)

D1 · Primary protocol

Approved: repeated grouped 5-fold CV (5 × 5 = 25 splits), no separate hold-out, small fixed model menu.
Alternative: a grouped 20% hold-out locked away until the end, plus CV on the rest. Cleaner in principle, but the hold-out would hold about one baby each for *scared* and *cold_hot*.

D2 · Classes

Approved: **all 8 classes**. Classes with fewer than 5 babies (*scared*, *cold_hot*) are kept in every metric but flagged "not reliably estimable" rather than given per-class conclusions.

D3 · Scope of deep learning

Approved: own small CNN (07) as the main DL model, with pretrained embeddings (07b) as an optional stretch. Colab can download the weights.
Alternative: CNN only, or pretrained embeddings only.

D4 · What gets committed

Approved: commit notebooks *with* their outputs (so GitHub shows the figures and numbers), `metadata/`, and small `results/` files. Never audio, caches or model weights.

D5 · Shared code

Approved: reusable functions in `src/` with pytest tests, imported by notebooks, as described in 4.3.
Alternative: everything inline in notebooks (simpler to read alone, but duplicated code and no tests).

D6 · Where deep learning runs

Chosen: Google Colab free tier with a T4 GPU, data from Google Drive, resumable per-split checkpoints (section 7.1). Pay As You Go or Kaggle only as a fallback.

> **Approved.** You accepted D1–D5 as recommended and asked for the CNN to run on Google Colab's GPU (D6). The build starts with chunk 1.